Research Activity: Alternating Least Squares

This report describes Alternating Least Squares (ALS) as a matrix-factorization approach for collaborative filtering and outlines a 10-step procedure for training explicit-feedback ALS on a star-rating dataset. Section A addresses eight key foundational questions. Section B presents the algorithm design and a flowchart. The report is limited to design: it does not include code, screenshots, or evaluation results.

Section A: Foundational research questions

A1. What ALS stands for and the problem it solves

What is ALS? The short answer: ALS is a matrix factorization method for collaborative filtering. It's used to solve the "rating-prediction" problem, in which you start with a large, sparse matrix R of user-item ratings (most users have rated a tiny portion of the items), and for each missing rating, try to predict it so you can tell what to recommend to the user [1] [2]. It does this by simultaneously learning a vector representation for every user and every item, so the rating is approximated by the dot product of the two vectors. It's popular because each training iteration has a closed form, easily parallelizable solution, which is why it ships as Spark's built-in collaborative-filtering algorithm [1].

A2. Matrix factorization, user factors, item factors and k

Matrix factorization approximates the m × n rating matrix R (m users, n items) as the product of two thin matrices:

$$R \approx U V^T,\ U \in \mathbb{R}^{mxk},\ V \in \mathbb{R}^{nxk},\ \hat{r}_{ui} = u_u{}^T v_i$$

- User factors (U): row u is a k-dimensional vector where u is attracted to each of the factors.
- Item factors (V): row i is a k-dimensional vector indicating in which strengths item i shows all the latent dimensions.
- Latent-factor dimension k (the rank): the number of hidden factors. (Though this is not preset – k is learned, and the learned k sometimes loosely corresponds to genre, mood, production style, etc.) k is much smaller than m or n (e.g., 10–200), so the model stores (m + n) × k numbers, rather than m × n.

As an example, 6,040 users and 3,706 movies with k = 20 require U to be 6,040 x 20 and V to be 3,706 x 20 – their product predicting all 6,040 x 3,706 predictions.

A3.Why the method is called "alternating"

The joint objective in U and V is non-convex because predictions multiply two unknowns. ALS breaks this by alternating: fix V and solve for U, then fix U and solve for V, and repeat [2]. When one matrix is held fixed, the prediction is linear in the other,
Hence the goal reduces to an ordinary regularized least-squares problem (i.e. Ridge regression) which admits an explicit solution. We can simply solve each vector for each user separately and independently of the other users:

$$\mathbf{u_u = (V_u^T V_u + \lambda\, n_u\, I)^{-1} V_u^T r_u}$$

The set of items user u rated, n_u = |I_u|, and r_{u, I_u} is those ratings. The item update is symmetrical. Note that each step can only decrease (or leave unchanged) the objective, so the loss is monotonically decreasing and convergence to a local minimum is guaranteed.

A4. The objective function, regularization and missing entries

Explicit ALS minimizes squared error only on the observed ratings and the L2 penalty on the factors,...

$$\mathbf{\min \text{ over } U, V: \Sigma_{(u,i)} \in \Omega\, (r_{ui} - u_u^T v_i)^2 + \lambda\, (\Sigma_u\, n_u\, \|u_u\|^2 + \Sigma_i\, n_i\, \|v_i\|^2)}$$

- Observed ratings (Ω): the only cells that go in the error term. They are the data the model is trying to fit.
- Loss term: the distance between the prediction and the existing rating.
- Regularization (λ): discourages large factor values so that the model doesn't memorize the small number of ratings for each user. Scaling the penalty according to the number of ratings for each user, n u, and each item, n i, (the weighted-λ form as used in Spark) allows a single λ to work for users with very different activity levels [1].

Missing values cannot be replaced by zeros. Zero means "zero stars" not "not rated". If 0's were used instead, the algorithm would learn to "hate" just about everything, all predictions would be driven to zero, and the sparse real information (~1 to 5% of cells [2]) would be drowned out.

A5. When ALS is useful, compared with neighborhood-based CF

Aspect	ALS (matrix factorization)	Neighborhood-based CF (user-kNN / item-kNN)
Sparsity	Generalizes through shared latent factors even when two users share no rated items	Needs overlapping ratings to compute similarity; weak on very sparse data
Scalability	Updates are independent per user and per item, so they parallelize well [1]	Pairwise similarity is costly for large catalogs
Prediction cost	One dot product of length k	Search and aggregate over neighbors
Memory	(m + n) × k parameters	Similarity matrix can approach n × n
Interpretability	Latent factors are hard to explain	"Because you liked X" is easy to explain
New ratings	Usually needs retraining or a fold-in step	New ratings can be used immediately
Hyperparameters	Rank, λ and iterations must be tuned	Mainly neighborhood size and similarity metric

ALS is preferred for large, sparse data sets on distributed systems. Neighborhood methods are still good for small data sets and if you care about interpretability.

A6.Explicit versus implicit feedback

Explicit feedback is expressed preference (e.g. A 1 to 5 star ranking); it has both positive and negative signals but is rare. Implicit feedback is action (e.g. click, view, or play count); it is plentiful but only signifies interest indirectly, and a missing interaction may mean dislike or simply that the user never saw the item.

The implicit-feedback formulation alters the objective in three ways [1], [2]:

1. Counts are converted into a binary preference p_ui (1=if any interaction, 0=if no interaction).
2. Every cell, whether observed or not, is included in the loss, because absence is weak evidence of non-preference.
3. The value c_ui = 1 + α r_ui is a confidence weight for each cell, so repeated interactions weigh more heavily.

$$\textbf{min over U, V: } \Sigma_{u,i}\, c_{ui}\, (p_{ui} - u_u^T v_i)^2 + \lambda\, (\Sigma_u \|u_u\|^2 + \Sigma_i \|v_i\|^2)$$

The extra hyperparameter α controls how fast confidence grows. In Spark this mode is enabled with the implicitPrefs and alpha parameters [1].

A7. Hyperparameters and their effects

Hyperparameter	Too small	Too large
Rank k	Underfits: too few dimensions to capture tastes, high training and validation error	Overfits sparse users, more memory and slower solves (each solve inverts a k × k matrix)
Regularization λ	Overfits: training error low, validation error high	Underfits: factors shrink toward zero and predictions flatten toward the mean
Iterations	Stops before convergence, error still falling	Little gain after convergence, wasted compute
α (implicit only)	Interactions barely weighted above non-interactions	Heavy users dominate the loss

Rank and λ: larger rank often requires larger λ and they are tuned together on a validation set. One can try Spark defaults as a starting point (rank 10, regParam 0.1 and maxIter 10, but these are not necessarily the tuned values [1]).

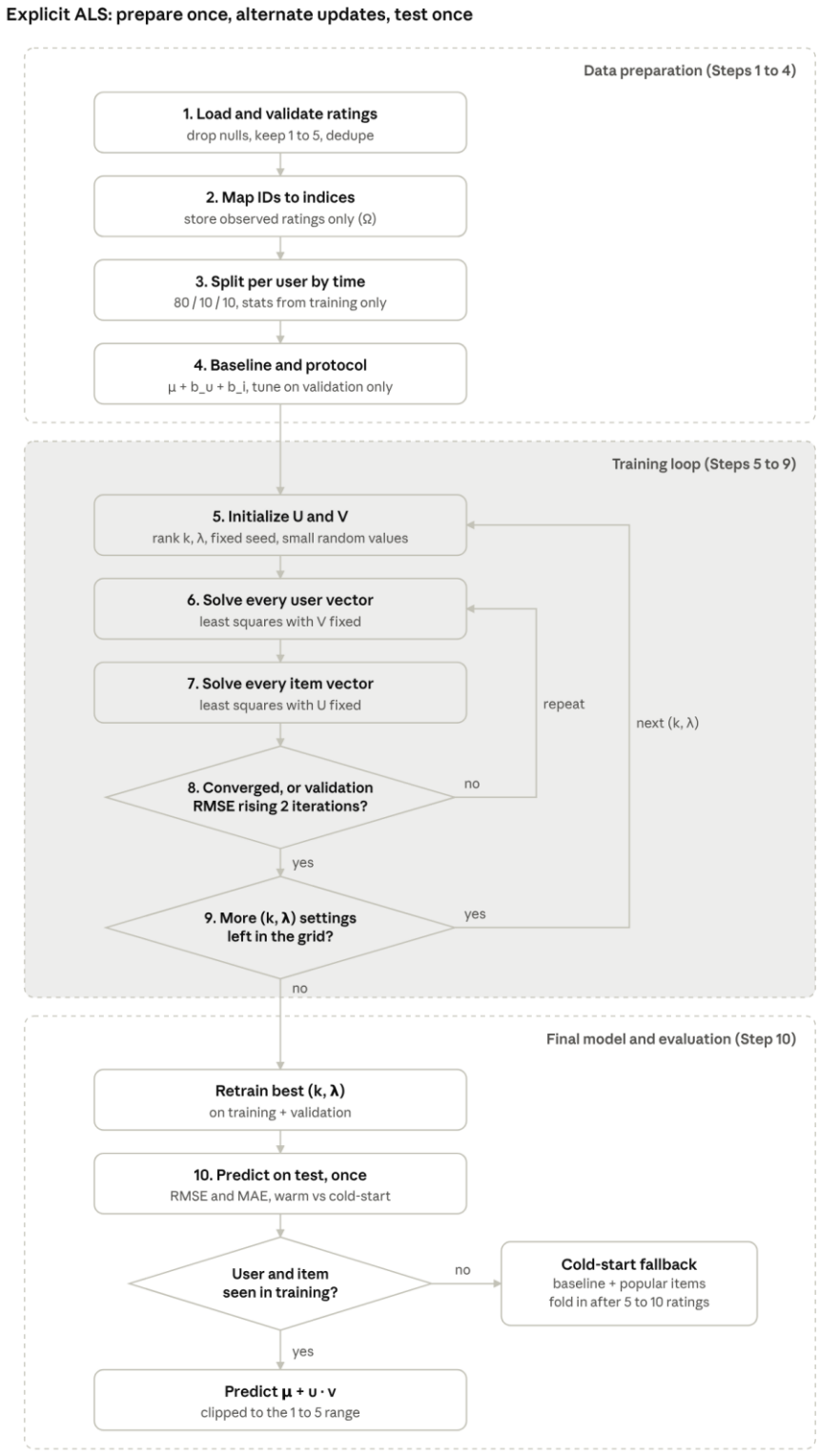
A8. Limitations and a fallback for new users

- Cold start: When a user or item is new, ALS will have no ratings to learn the corresponding factor vector. Spark does not score such pairs, but provides a coldStartStrategy option to discard them when evaluating [1].
- Popularity imbalance: A few popular items may take the majority of the ratings, so their factors are well estimated and they are recommended too frequently, while long-tail items hardly appear.
- Sparsity: having only a few ratings causes users to have very regularized, unstable vectors, thus their predictions are near the mean.

Fall back suggested for a brand-new user: a bias baseline prediction (i.e. Global mean+item bias), and recommend popular, high-rated items across any category or demographic provided at sign-up. After the user has rated ~5-10 items, calculate their factor vector by "folding in": solve the single user equation from A3 with V fixed, without retraining the whole model.New items operate identically reversed, and those content features (genre, description) may provide an initial rough estimate until ratings come in [2].

Section B: Implementation plan for explicit-feedback ALS

The plan trains explicit ALS in 10 steps on a 1 to 5 star data set (We'll assume MovieLens 1M, but any user, item, rating, timestamp table is fine). The training loop is steps 5 to 8. Step 9 repeats the training loop for each hyper-parameter. Here is the flow chart of what we will be doing.



Explicit ALS implementation flowchart {10 steps, 3 decisions}

Within the inner loop (Steps 6-8), the two least-squares updates are interleaved until training converges; in the outer loop (Steps 5-9), training is repeated for each hyperparameter setting, and the test set is used only once at the end.

Step 1. Load and validate the ratings

- Purpose: Use a sound and reliable collection of observed ratings.
- Input: raw ratings file with userId itemId rating timestamp.
- Output: Completed cleaned table of data: (user, item, rating, timestamp).
- Decision/check: if any field is missing, drop line; discard rating if rating less than 1 or greater than 5; if any user rate any item more than 1 time, retain the latest rating with the later timestamp.

Step 2. ID and separate observed from missing.

- Purpose: store the matrices using row indices in a compact fashion and only output real rating values.
- Input: cleaned table from Step 1.
- Output: user-index map (raw ID to 0..m-1), item-index map (raw ID to 0..n-1) and a sparse list Ω of observed (u, i, r) triples sorted by user and sorted by item.
- Decision/check: maps are one-to-one and counts match with the ids; unrated cells are not stored and not filled as zero (see A4); For confirmation that data is sparse record the density $|\Omega| / (m \times n)$.

Step 3. Split into train, validation and test sets without leakage

- Purpose: estimate ratings that it has not been trained on.
- Input: Ω with time-stamps.
- Output: per-user chronological split: For each user, put the oldest 80% of ratings in the training set, the next 10% in the validation set, and the most recent 10% in the test set.
- Decision/check: no (user, item) pair appears in two sets; all statistics used later (global mean, user biases, item biases, user-to-training-item map, item-to-training-user map) are computed from the training set only; validation and test pairs whose user or item does not exist in training are labeled as cold-start and routed to Step 10's fallback.

Step 4. Fix the evaluation protocol and a baseline

- Purpose: set a goal in advance of the training so that the success cannot be cheatable by the test set.
- Input: training and validation sets.
- Outputs: global average μ, user biases b u, item biases b i from training; baseline estimate $\mu + b_u + b_i$ and its validation RMSE and MAE.

- Decision/check: all tuning uses validation; test set is accessed exactly once, in step 10; the only time ALS is accepted is if it outperforms this baseline on validation.

Step 5. Choose Your Dimensions, Initialize and Set Parameters

- Purpose: initialize the two factor matrices and define the search space.
- Input: m, n and the hyperparameter grid: rank k in {10, 20, 50, 100}, λ in {0.01, 0.05, 0.1, 0.5} maximum 20 iterations, tolerance 10^-4.
- Output: U (m × k) and V (n × k) initialized with small random values (for example normal with standard deviation 0.01) with a fixed random seed.
- Decision/check: everything looks ok, no NaNs, shapes are ok, ratings are centered by subtracting μ – so model deviations from the mean.

Step 6. Update all user factors with V fixed

- Purpose: first half of 1st alternating iteration.
- Input: current V and each user's training ratings.
- Output: new U with rows solving the regularized least-squares problem in A3.
- decision/check: resolve for each user independently (parallelisable); $\lambda > 0$ ensures that all the k×k systems are invertible; a user with no training ratings is simply skipped and goes along the cold-start route.

Step 7. Update all item factors with U fixed

- Purpose: The rest of the iteration.
- Input: the new U and the training scores for each item.
- Output: New V produced, itemised, with the symmetric equation.
- Decision/check: same invertibility and empty-item checks as Step 6, Step 6 and Step 7 comprise one iteration.

Step 8. Track error and determine if terminate

- Purpose: to determine the convergence and overfitting.
- Input: U and V after the ith iteration for the training and validation set.
- Output: training loss and validation RMSE at each iteration, and the current best (U, V).
- Decision/check: loss should never go up (if it does it is a bug in the implementation) end when relative of the decrement of the loss goes below 10^(-4), while validation rmse goes up 2 times in a row or you hit the iteration limit; otherwise go back to step 6.

Step 9. Tune hyperparameters on validation

- Purpose: provide the rank and regularization parameters that generalize optimally.
- Input: the grid from Step 5.
- Output: the best (k, λ, iterations) combination with respect to validation RMSE, and a final model retrained on training+validation with those parameters.
- Decision/check: if the optimal value is at the edge of the grid, extend in that direction and repeat; then show the validation curve for every setting.

Step 10. Predict, evaluate for single time and cold start.

- Purpose: to generate the ratings and the ultimate, unbiased performance estimate.
- Input: Final model, test set, cold-start pairs of Step 3.
- Predictions: $\hat{r}_{ui} = \mu + u_u \cdot v_i$, or value clip to 1 5 and test RMSE / MAE reported for known pairs and cold start pairs separately.
- Decision/check: for unknown user/item, output bias baseline ($\mu + b_i$ for new user, $\mu + b_u$ for new item) and recommend popular top-rated items; After getting around 5 to 10 ratings, include them by solving the Step 6 equation for that user only with V fixed.

$$\mathbf{RMSE} = \sqrt{\left(\frac{1}{|T|} \right) \sum_{(u,i) \in T} (r_{ui} - \hat{r}_{ui})^2}$$

$$\mathbf{MAE} = \left(\frac{1}{|T|} \right) \sum_{(u,i) \in T} |r_{ui} - \hat{r}_{ui}|$$

T is the test set. RMSE punishes large errors more heavily, while MAE gives the average error in stars.

References

[1] The Apache Software Foundation, "Collaborative Filtering," *Apache Spark MLlib Guide*, latest release. [Online]. Available: https://spark.apache.org/docs/latest/ml-collaborative-filtering.html. See also the ALS API reference: https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.recommendation.ALS.html. Accessed: Oct. 29, 2026.

[2] Y. Hu, Y. Koren, and C. Volinsky, "Collaborative Filtering for Implicit Feedback Datasets," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, Pisa, Italy, 2008, pp. 263–272, doi: 10.1109/ICDM.2008.22.